

VexIR Instruction set

1) Constants

1. `CONST_I32 t1 (number)`
2. `CONST_BOOL t1 (true/false)`
3. `CONST_STR t1 (string value)`

2) Memory

1. `ALLOC_LOCAL x1` - Here, `x1` represents the address of a variable
2. `STORE x1 t1`
3. `LOAD t1 x1`

3) Arrays

1. `ALLOC_ARRAY a1 5` - Array initial address `a1` , allocates an array of size `5`
2. `ELEM_ADDR t2 a1 t1` - Store address of `a1[t1]` in `t2`

4) Unary Ops

1. `NEG t2 t1`
2. `NOT t2 t1`
3. `MOV t2 t1`

5) Binary Ops

1. `ADD t3 t1 t2`
2. `SUB t3 t1 t2`
3. `MUL t3 t1 t2`
4. `DIV t3 t1 t2`
5. `REM t3 t1 t2`
6. `AND t3 t1 t2`
7. `OR t3 t1 t2`
8. `GT t3 t1 t2`
9. `GEQ t3 t1 t2`
10. `LT t3 t1 t2`
11. `LEQ t3 t1 t2`
12. `EQ t3 t1 t2`
13. `NEQ t3 t1 t2`

6) Control Flow

1. `LABEL (label name)`
2. `JUMP (label name)`
3. `BRANCH t1 (label1 name) (label2 name)`
4. `RETURN t1`

5. RETURN_VOID - Return no value

7) Calls / Functions

1. CALL t1 (func_name) t1 t2 (more args...) - Call with return value
2. CALL_VOID (func_name) t2 t3 (more args...) - Call without return value
3. To define functions:

```
FUNCTION (func_name)
PARAM p1
PARAM p2
(...)
FUNC_START
...
FUNC_END
```

Note: Type checking will already be handled during semantic analysis, so ideally all code in IR is type-safe. Thus, we don't have explicit types of temporaries or variables.